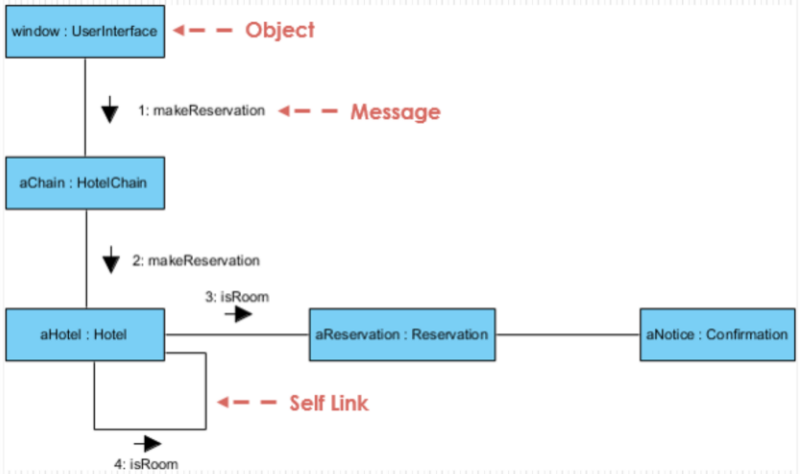
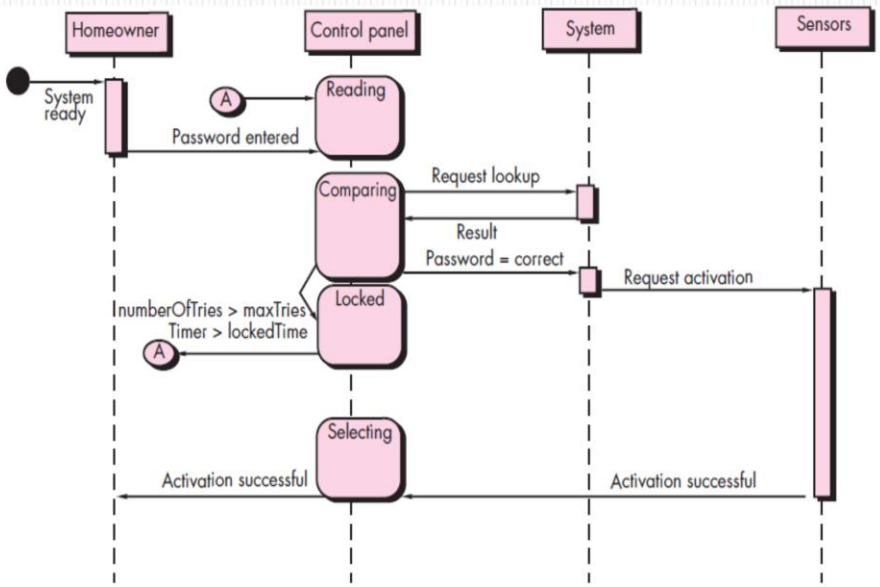
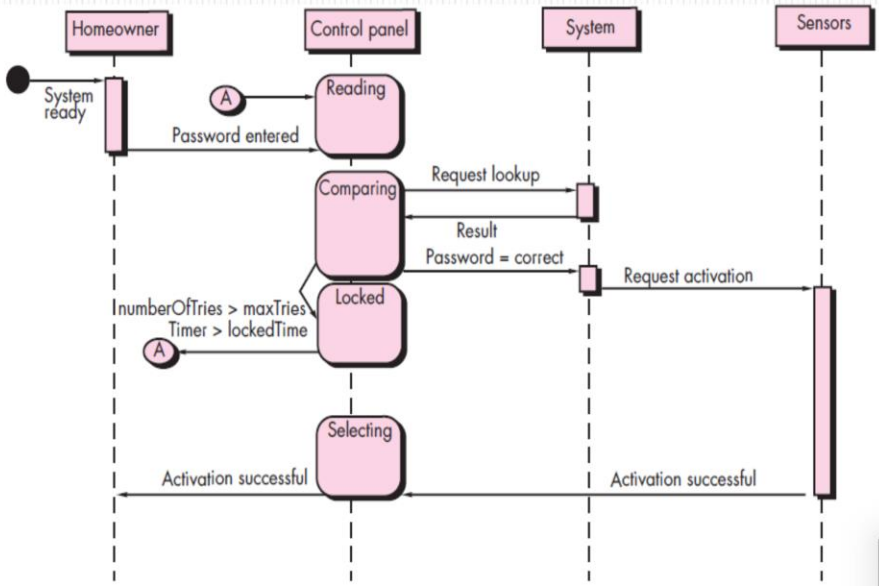
Collaboration/Communication Diagram



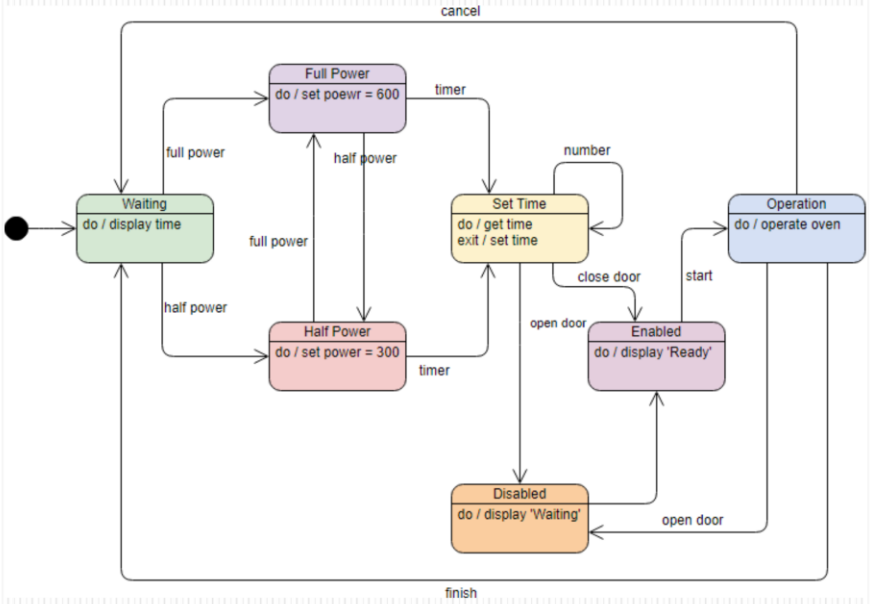
Sequence diagram

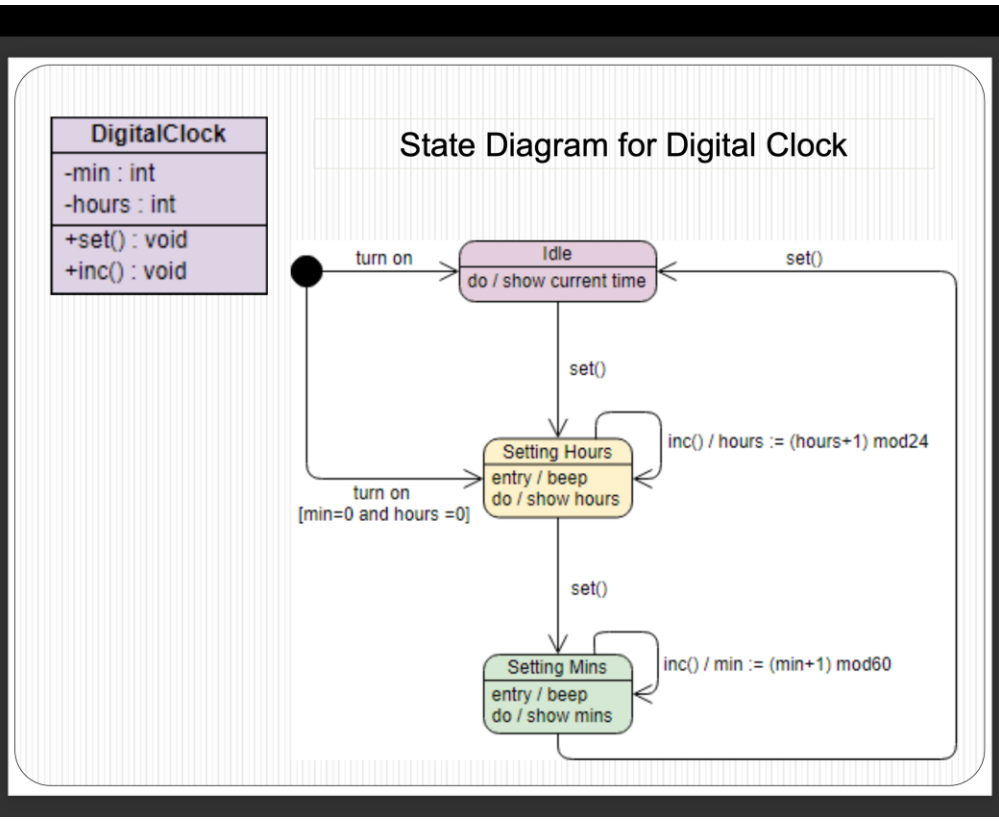


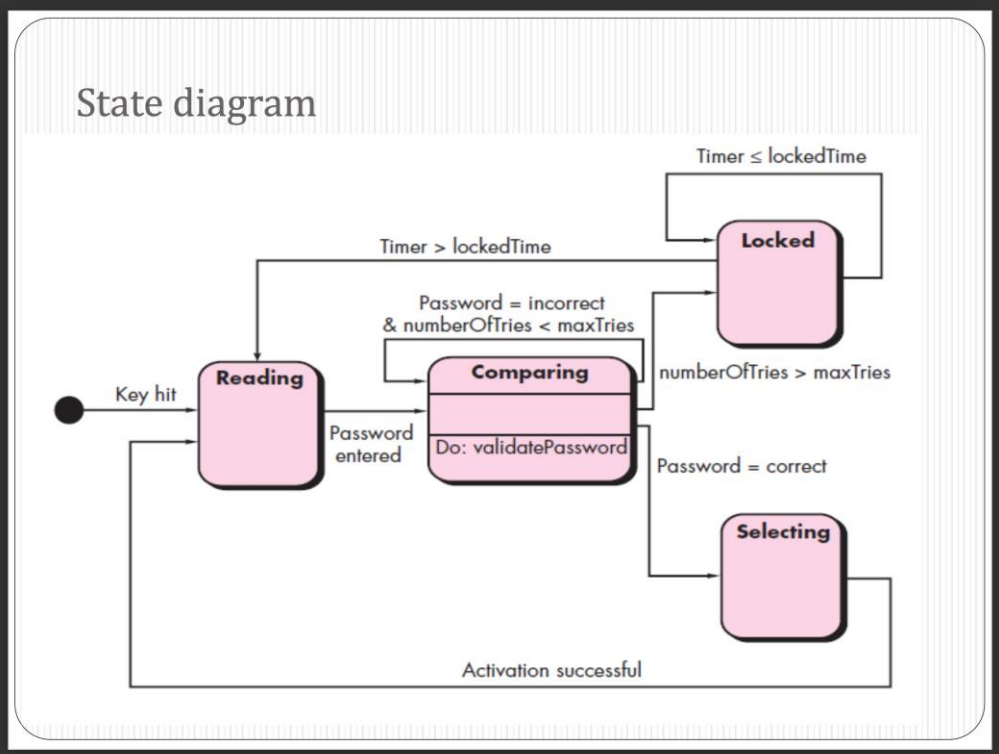
Sequence diagram



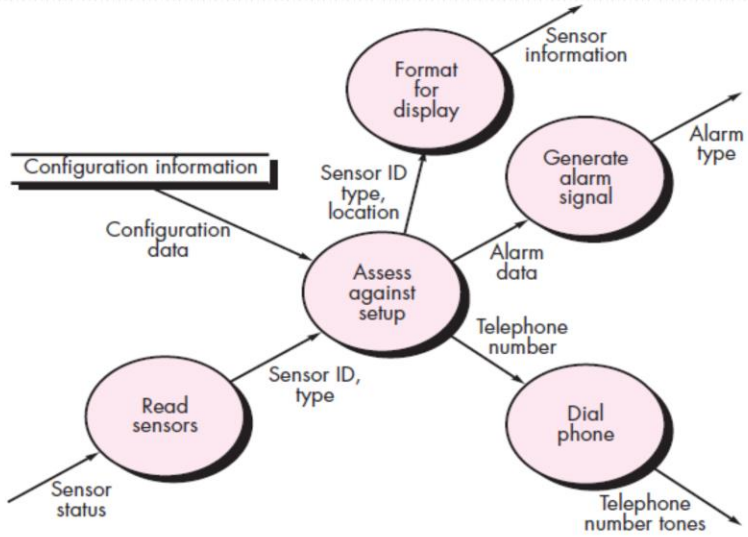
State Diagram for Oven



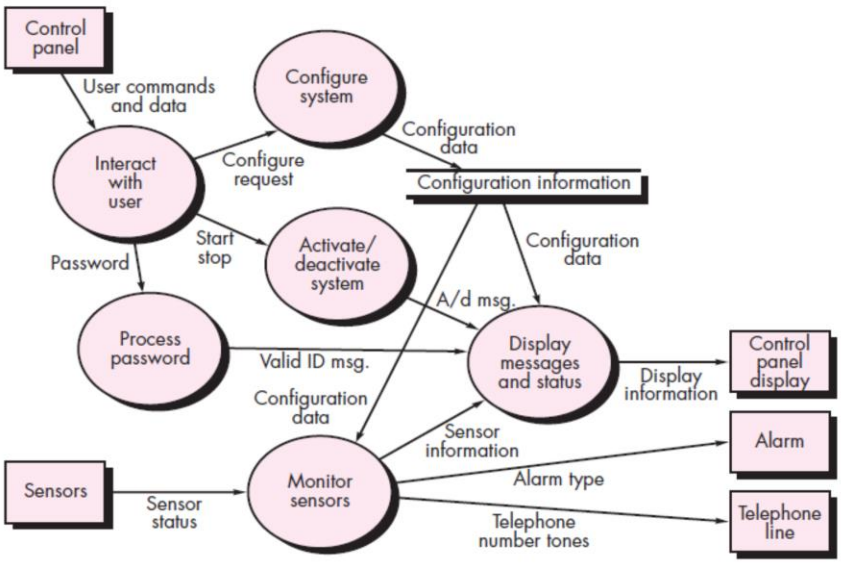

DigitalClock
-min : int
-hours : int
+set() : void
+inc() : void
State Diagram for Digital Clock
turn on
Idle
do / show current time
set()
set()
Setting Hours
entry / beep
do / show hours
inc() / hours := (hours+1) mod24
turn on
[min=0 and hours =0]
set()
Setting Mins
entry / beep
do / show mins
inc() / min := (min+1) mod60


State diagram
Timer ≤ lockedTime
Locked
Timer > lockedTime
Password = incorrect
& numberOfTries < maxTries
Reading
Comparing
Do: validatePassword
numberOfTries > maxTries
Key hit
Password
entered
Password = correct
Selecting
Activation successful

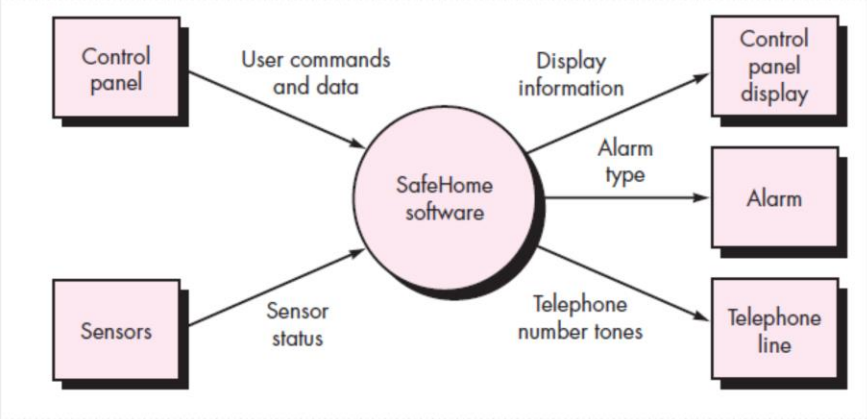
LEVEL – 2 DFD



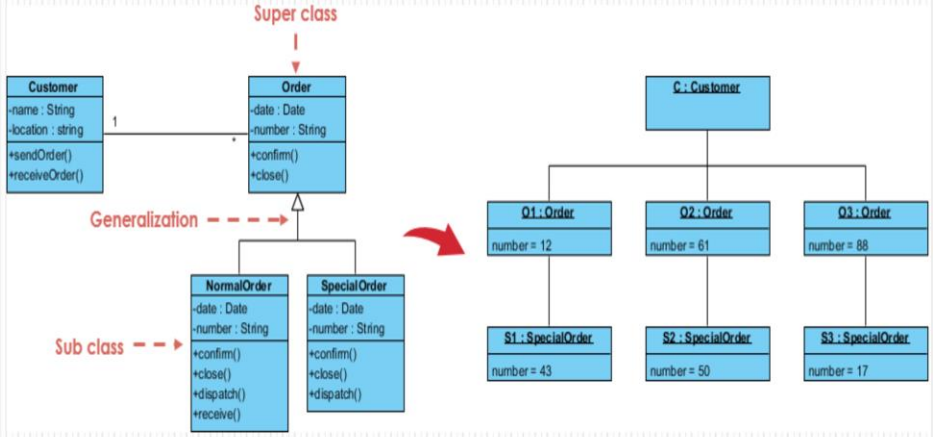
LEVEL – 1 DFD

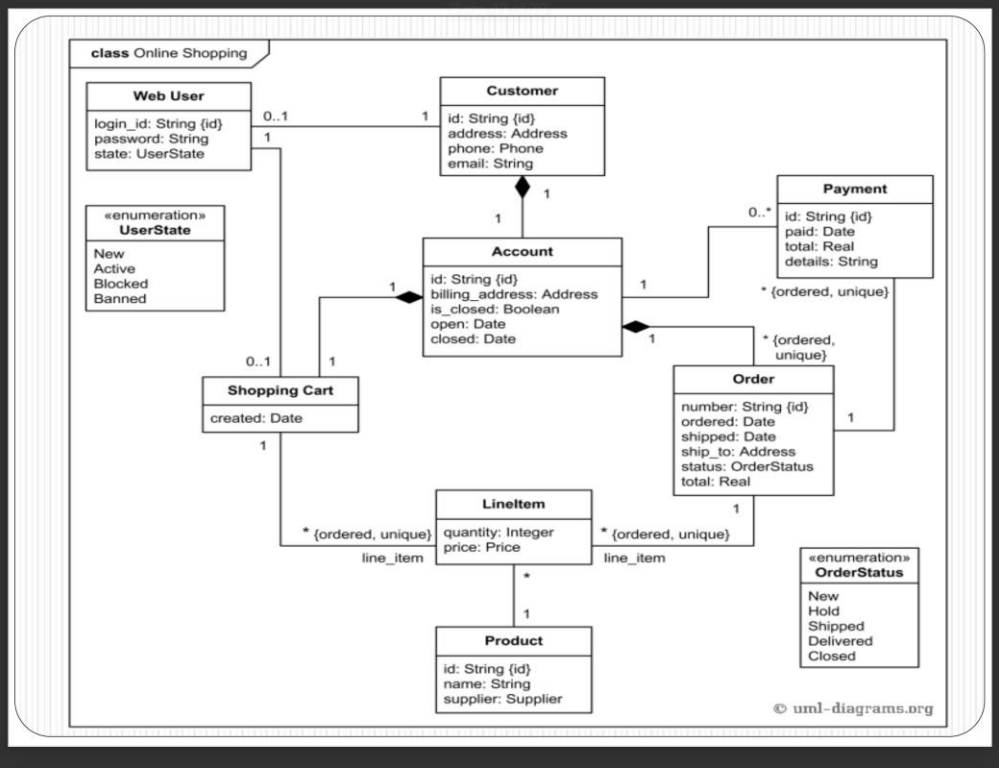


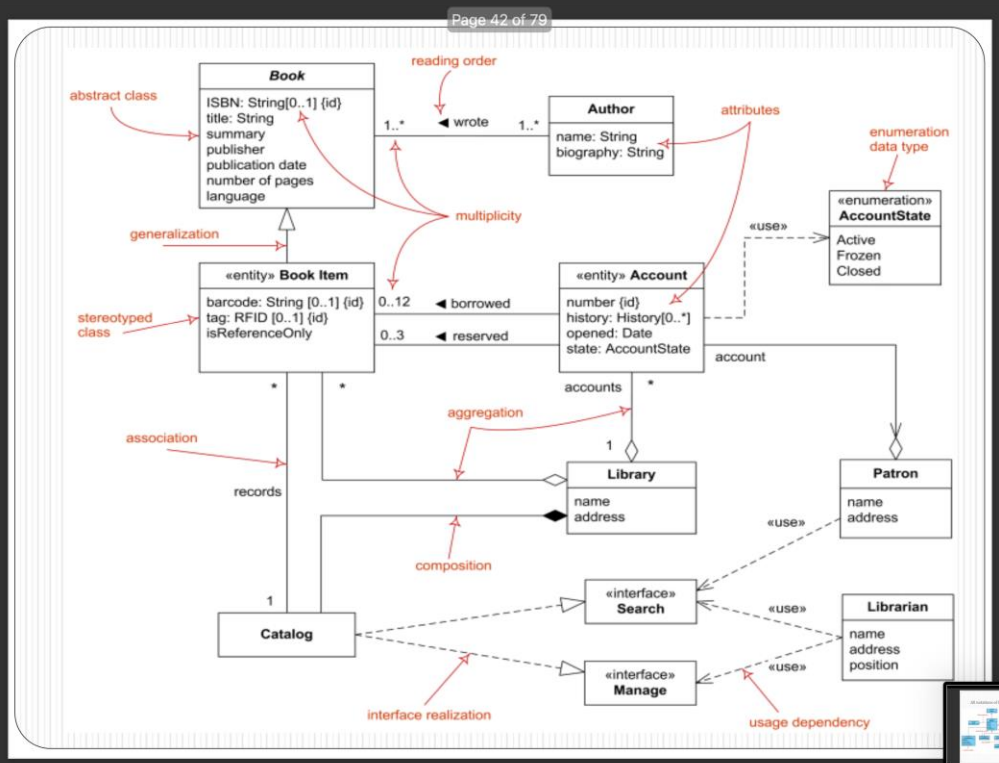
Level – 0 DFD / context level diagram



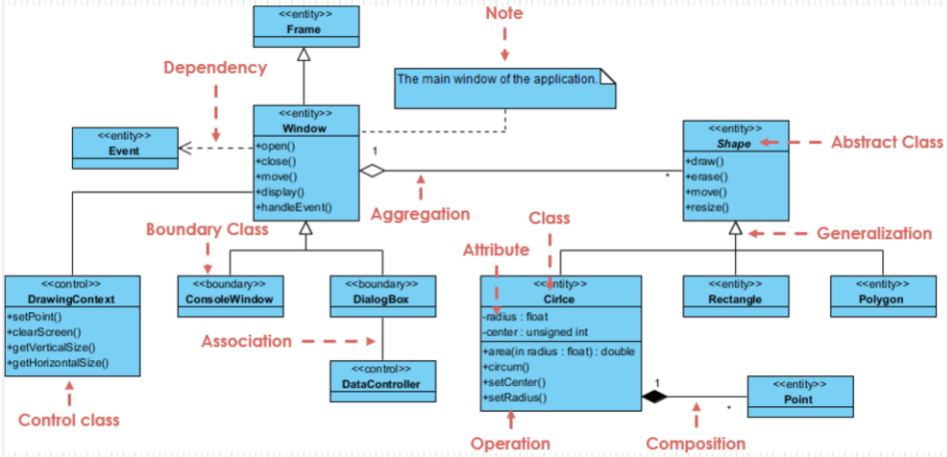
Object Diagram




class Online Shopping
Web User
login_id: String {id}
password: String
state: UserState
Customer
id: String {id}
address: Address
phone: Phone
email: String
«enumeration»
UserState
New
Active
Blocked
Banned
Payment
id: String {id}
paid: Date
total: Real
details: String
Account
id: String {id}
billing_address: Address
is_closed: Boolean
open: Date
closed: Date
* {ordered, unique}
* {ordered, unique}
Shopping Cart
created: Date
Order
number: String {id}
ordered: Date
shipped: Date
ship_to: Address
status: OrderStatus
total: Real
LineItem
quantity: Integer
price: Price
* {ordered, unique}
line_item
* {ordered, unique}
line_item
«enumeration»
OrderStatus
New
Hold
Shipped
Delivered
Closed
Product
id: String {id}
name: String
supplier: Supplier
© uml-diagrams.org

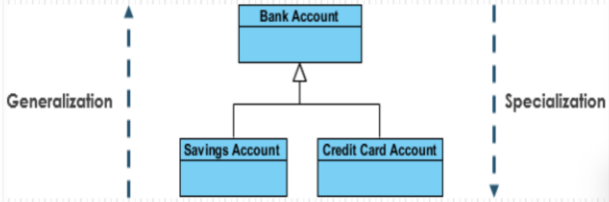

Page 42 of 79
abstract class
Book
ISBN: String[0..1] {id}
title: String
summary
publisher
publication date
number of pages
language
reading order
wrote
Author
name: String
biography: String
attributes
enumeration
data type
«enumeration»
AccountState
Active
Frozen
Closed
multiplicity
generalization
«use»
«entity» Book Item
barcode: String [0..1] {id}
tag: RFID [0..1] {id}
isReferenceOnly
borrowed
reserved
«entity» Account
number {id}
history: History[0..*]
opened: Date
state: AccountState
stereotyped
class
account
accounts
aggregation
association
records
Library
name
address
Patron
name
address
composition
«interface»
Search
Librarian
name
address
position
Catalog
«interface»
Manage
interface realization
usage dependency

All notations of Class Diagram



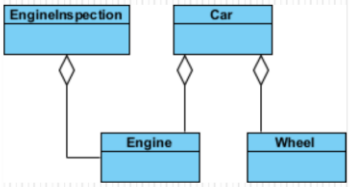
Generalization vs. Specialization

- **Generalization** identifies commonalities among a set of entities. The commonality may be of attributes, behavior, or both. A subclass may have more specialized attributes and operations.
- **Specialization** is the reverse process of Generalization means creating new sub classes from an existing class.
- For Example, a Bank Account is of two types - Savings Account and Credit Card Account. Savings Account and Credit Card Account inherit the common/ generalized properties like Account Number, Account Balance etc. from a Bank Account and also have their own specialized properties like unsettled payment etc.

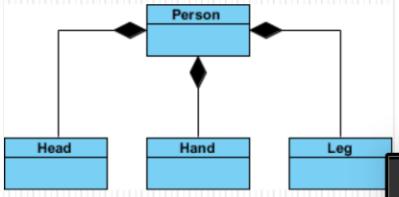


Aggregation vs. Composition

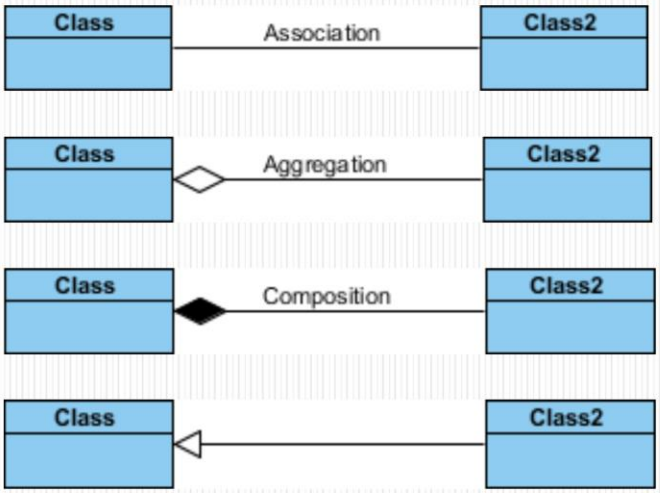
- **Aggregation** implies a relationship where the child can exist independently of the parent. Example: Class (parent) and Student (child). Delete the Class and the Students still exist.



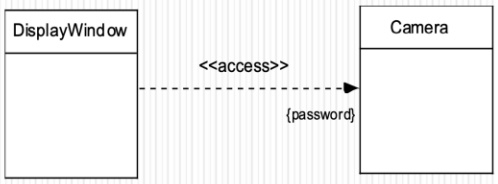
- **Composition** implies a relationship where the child cannot exist independent of the parent. Example: House (parent) and Room (child). Rooms don't exist separate to a House.



Relationships

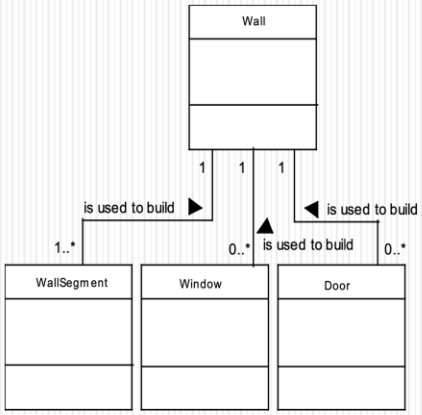


Dependencies

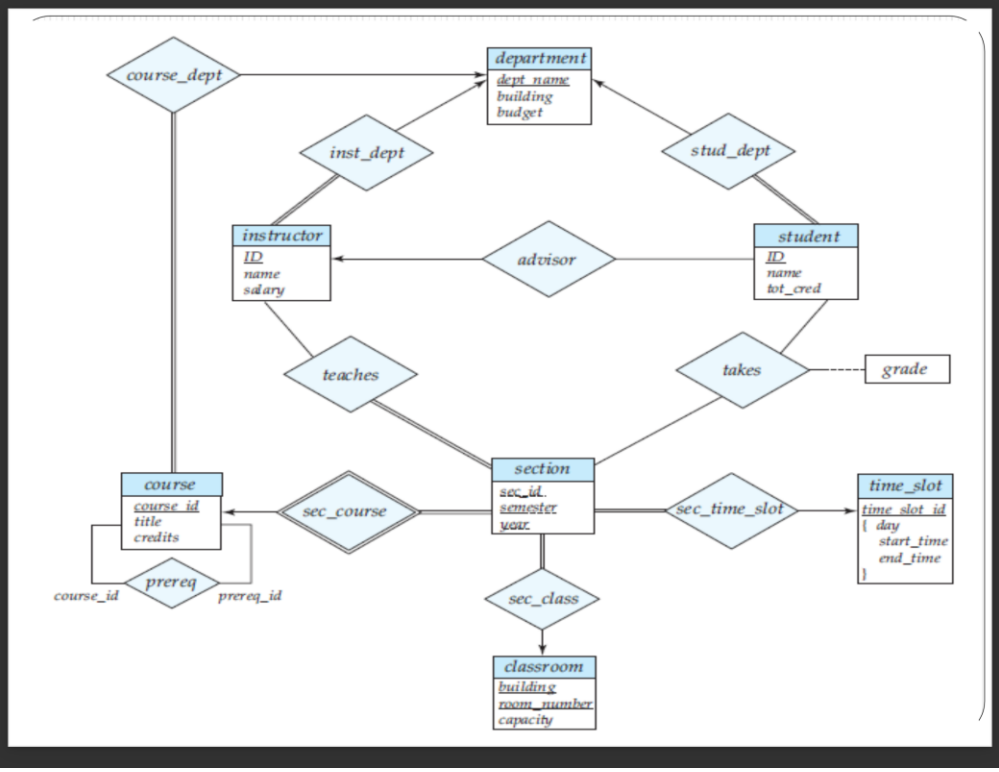


Usage is a dependency in which one **named element** (client) requires another named element (supplier) for its full **definition** or **implementation**.

Multiplicity

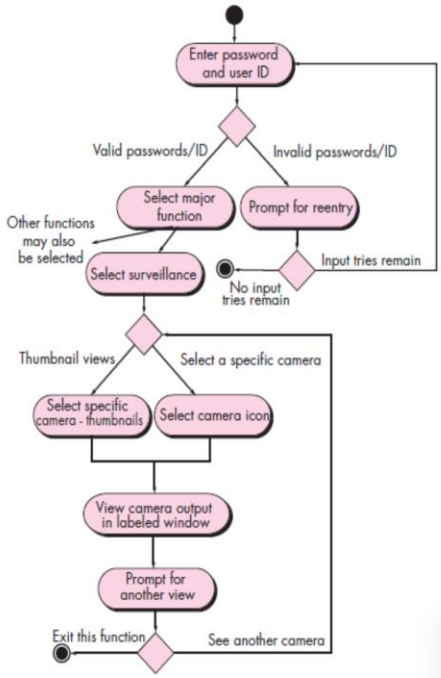


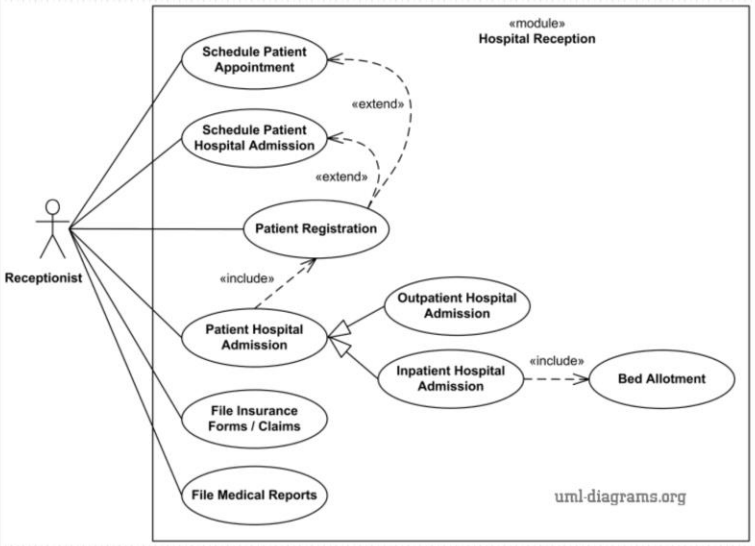
Indicator	Meaning
0..1	Zero or one
1	One only
0..*	Zero or more
1..*	One or more
n	Only *n* (where *n* > 1)
0..n	Zero to *n* (where *n* > 1)
1..n	One to *n* (where *n* > 1)



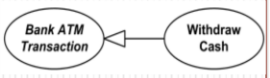
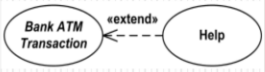
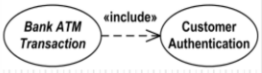
Activity Diagram

Supplements the use case by providing a graphical representation of the flow of interaction within a specific scenario





Relationships

Generalization	Extend	Include
		
Base use case could be abstract use case (incomplete) or concrete (complete).	Base use case is complete (concrete) by itself, defined independently.	Base use case is incomplete (abstract use case).
Specialized use case is required, not optional, if base use case is abstract.	Extending use case is optional, supplementary.	Included use case required, not optional.
No explicit location to use specialization.	Has at least one explicit extension location.	No explicit inclusion location but is included at some location.
No explicit condition to use specialization.	Could have optional extension condition.	No explicit inclusion condition.